

SparkQuest: A Gamified, Browser-Based System for Teaching Python, PySpark, and Spark Structured Streaming with an AI Tutor and Real-Execution Auto-Grading

saipavan333

Independent Project | github.com/saipavan333/sparkquest

ABSTRACT

Abstract—Distributed data processing with Apache Spark is a core competency in modern data engineering, yet beginners face a steep setup and conceptual barrier: installing a JVM and Spark, understanding lazy evaluation, and reasoning about distributed execution. We present *SparkQuest*, an open-source, browser-based learning system that takes a learner from elementary Python to PySpark and Spark Structured Streaming through 16 auto-graded, gamified challenges. Unlike multiple-choice tutorials, SparkQuest executes every submission against a *real* Spark engine in a process-isolated sandbox and grades it with a declarative, output-based checker, giving learners authentic feedback. An AI tutor provides Socratic hints with a fully offline rule-based fallback. We describe the system architecture, the security model of the execution sandbox, the declarative grading framework, and a finance-themed curriculum. We evaluate the engine on a constrained 2-vCPU host, reporting aggregation throughput up to 4.4×10^6 rows/s, streaming ingestion of $\sim 12,000$ events/s, sub-millisecond grading for pure-Python tasks, and a ~ 3.5 s warm-up amortized across a full Spark grade (~ 8.6 s end-to-end). All 16 reference solutions pass their own graders (100%), and the system ships with CI, containerization, and one-command deployment to Hugging Face Spaces.

Index Terms—Apache Spark, PySpark, Structured Streaming, data engineering education, automated assessment, gamification, sandboxed code execution, MLOps.

I. INTRODUCTION

Apache Spark [2] underpins a large fraction of industrial data pipelines, and fluency in PySpark and Spark Structured Streaming [4] is routinely required of data-engineering candidates. However, the on-ramp is unusually steep for newcomers. A learner must (i) install a compatible JVM and Spark distribution, (ii) internalize *lazy* transformation/action semantics that differ from eager Python, and (iii) reason about distributed execution, shuffles, and event-time processing. Traditional text- or video-based courses defer hands-on practice until after this setup tax is paid, and quiz-style “interactive” courses rarely run the learner’s actual code.

We argue that an effective introduction should let a beginner write and *execute* real Spark code in the first minutes, with immediate, trustworthy feedback, and should sustain motivation through game mechanics. We realize this in **SparkQuest**, an open-source system with the following contributions:

- A *real-execution* learning platform: every submission runs against an actual Spark engine, not a simulation, inside a process-isolated sandbox with a hard timeout and output

caps (§A).

- A *declarative auto-grader* whose checks compare program state and DataFrame contents (order-insensitively) rather than brittle string matching (§B).
- A *zero-to-pro* curriculum of 16 finance-themed challenges spanning Python, PySpark/Spark SQL, and Structured Streaming (§IV), with gamification (XP, levels, badges, leaderboard).
- An *AI tutor* with pluggable LLM back-ends and an offline rule-based fallback, so the public demo is useful at zero cost (§A).
- An end-to-end, reproducible engineering pipeline—tests, CI, Docker, and one-command deployment—plus an empirical evaluation of the engine (§VI).

II. RELATED WORK

Spark and its programming model. Spark introduced resilient distributed datasets (RDDs) [1] as a fault-tolerant in-memory abstraction; Spark SQL [3] added the DataFrame API and the Catalyst optimizer, and Structured Streaming [4] unified batch and stream processing under one declarative API. The DataFrame group-by aggregation we benchmark traces conceptually to MapReduce [5], which we use pedagogically as a “word-count” bridge from plain Python to Spark.

Automated assessment. Automated grading in computing education is well studied [8]; test-based feedback can move students from trial-and-error toward reflection [9], and automation scales formative feedback [10]. Environments such as Greenfoot [11] lower setup friction for novices. SparkQuest differs by grading *distributed* DataFrame and streaming results, comparing collected rows as multisets rather than matching console text.

Gamification. Gamification—“the use of game design elements in non-game contexts” [6]—has shown mostly positive, context-dependent learning effects [7]. We adopt lightweight mechanics (XP, levels, badges, leaderboard) tied to verified solves.

III. SYSTEM ARCHITECTURE

SparkQuest is a single-container web application. A FastAPI back-end serves a static single-page front-end (a Monaco editor, lesson navigator, console, and tutor panel) and exposes a small JSON API: `/api/tracks`, `/api/challenge/{id}`, `/api/run`, `/api/submit`, `/api/tutor`, and `progress/leaderboard` endpoints. Lessons are declarative YAML documents loaded into an in-memory catalog at start-up; the public catalog deliberately omits reference solutions and grader checks so they cannot be scraped by the client.

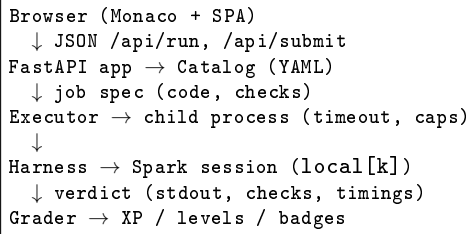


Figure 1. Request flow. Untrusted code is executed only inside the child process, never in the API process.

A. Process-Isolated Execution Sandbox

Each submission is executed in a freshly spawned child process (Fig. 1). The parent writes a JSON *job* (learner code, the challenge’s checks, Spark master, limits) and reads back only a JSON *verdict*; it communicates through files, so a hang, crash, or out-of-memory condition in learner code cannot corrupt the server. The parent enforces a wall-clock timeout (25 s for Python, 90 s for Spark) and truncates captured `stdout/stderr`. For Spark challenges the harness injects a tuned `SparkSession` (UI disabled, four shuffle partitions, Arrow enabled) bound to loopback via `SPARK_LOCAL_IP` so it never depends on resolving the container hostname—a common failure mode in sandboxes and PaaS runtimes that we encountered and fixed.

This MVP isolation (process boundary + timeout + output caps) is appropriate for a single-tenant educational demo but is explicitly *not* a hostile-multi-tenant sandbox. We document the production hardening path: execute each submission in an ephemeral, network-isolated container (e.g. gVisor or nsjail) and replace the per-submission `local[k]` session with a thin Spark Connect client to a warm, shared cluster, eliminating JVM cold-start.

B. Declarative Auto-Grader

A challenge declares a list of typed *checks* evaluated against the post-execution namespace. Supported checks include `stdout_contains`, `var_equals/var_close`, `callable_returns`, and DataFrame-aware checks `df_columns`, `df_row_count`, and `df_equals`. The latter collects a DataFrame and compares it to expected rows as a *multiset* (order-insensitive), which matches Spark’s non-deterministic row ordering. A custom check allows author-written assertions for stateful streaming results. A submission passes only if it runs cleanly *and* every check passes; passing awards difficulty-scaled XP. Listing 1 shows a representative PySpark challenge and its grader.

Listing 1. A PySpark group-by challenge: learner solution (top) and the declarative grader check (bottom).

```

totals = (trades.groupBy("ticker")
          .agg(F.sum("qty").alias("total")))

# grader check (YAML)
- type: df_equals
  name: totals
  expected:

```

Table 1. Curriculum overview (16 challenges, finance-themed).

Track	Representative skills	#
Python Foundations	variables, lists, dicts, functions, a “word-count” MapReduce bridge	6
PySpark & Spark SQL	DataFrames, select/filter, withColumn, group-by, joins, SQL views, windows	7
Structured Streaming	file source, stateful aggregation, event-time windows + watermarks	3

```

- {ticker: AAPL, total: 170}
- {ticker: GOOG, total: 70}

```

IV. PEDAGOGICAL & CURRICULUM DESIGN

The curriculum comprises 16 challenges in three ordered tracks, each challenge carrying a brief, starter code, hints, a reference solution, and grader checks.

Three design choices are central. First, *continuity of mental model*: the Python track culminates in a dictionary-based word count, which is then re-derived as a Spark `groupBy`, making distributed aggregation feel familiar. Second, *determinism for grading*: most challenges build their own small DataFrames inline, and streaming lessons use a bundled file source with the `Trigger.AvailableNow` mode so a streaming query drains all input and stops, yielding a deterministic in-memory table to grade. Third, *domain framing*: all data are finance-themed (tickers, trades, sectors), aligning with the data-engineering contexts where Spark is most used.

A. AI Tutor

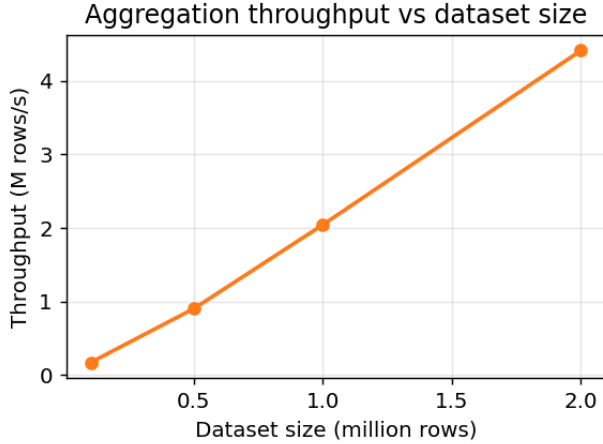
The tutor accepts the current code, the last error, and a learner question, and returns a concise, Socratic hint. It supports Anthropic, OpenAI, and Hugging Face back-ends selected by environment variables. When no key is configured it falls back to a deterministic rule-based tutor that pattern-matches common errors (e.g. `AnalysisException` → “check column names with `printSchema()`”) and otherwise surfaces the lesson’s next hint, so the public demo remains useful at zero cost and never blocks on an external service.

V. IMPLEMENTATION

The system is ~1,600 lines of Python and JavaScript: FastAPI and Pydantic for the API and typed models, PySpark 3.5.1 on OpenJDK 11 for execution, and a dependency-light vanilla-JS front-end embedding the Monaco editor. Progress is held in a thread-safe in-memory store with an interface that mirrors a persistent back-end (Redis/Postgres) for future scaling. The project is containerized (Python 3.10 + JRE 11 + PySpark), runs under a non-root user on port 7860 for Hugging Face Spaces, and includes GitHub Actions for linting, the full test suite, container publication to GHCR, and Space deployment.

Table 2. Group-by aggregation on `local [2]` (median of 2 runs).

Rows	Median time (s)	Throughput (rows/s)
100,000	0.588	170,168
500,000	0.554	903,014
1,000,000	0.489	2,043,617
2,000,000	0.453	4,413,262

**Figure 2.** Aggregation throughput vs. dataset size. Fixed job overhead amortizes as data grows.

VI. EXPERIMENTAL EVALUATION

Setup. All measurements were taken on a constrained host—Ubuntu 22.04, Python 3.10, OpenJDK 11, PySpark 3.5.1, **2 vCPU / 3.8 GiB RAM**—to reflect the free-tier environments learners and free PaaS deployments actually use. The benchmark harness is reproducible and optionally logs to Weights & Biases. Each data point is the median of repeated runs after a warm-up.

Correctness. Every one of the 16 reference solutions passes its own grader (100%), enforced both by a standalone validator and by a parameterized test in the CI suite, alongside unit tests for the sandbox (including timeout handling) and the API.

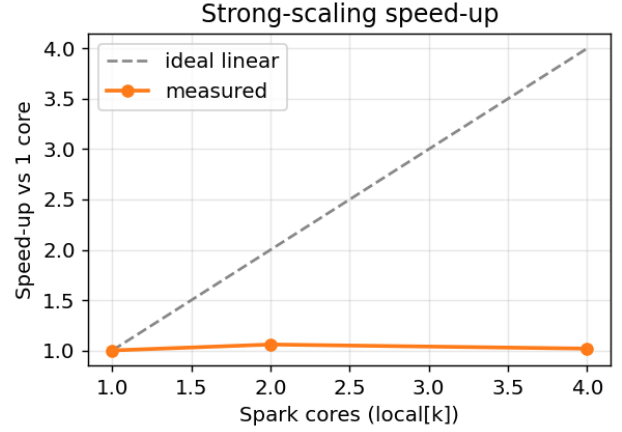
Aggregation throughput. Table 2 and Fig. 2 report a shuffle-heavy group-by aggregation at increasing sizes on `local [2]`. Wall-clock time is nearly flat ($\sim 0.45\text{--}0.59$ s) because a fixed per-job overhead (task scheduling, shuffle setup) dominates at this scale; consequently *throughput rises with size*, reaching 4.4×10^6 rows/s at 2 M rows as the fixed cost amortizes.

Strong scaling. Table 3 and Fig. 3 vary the worker count for a fixed 2 M-row job. Speed-up is modest ($1.06\times$ at two cores) and *regresses* at `local [4]` because the host has only two physical cores and the job is overhead-bound, leaving little to parallelize. This is the expected behavior for small jobs on small hardware and is revisited in §VII.

Streaming ingestion. A Structured Streaming query reading 100,000 events across 40 newline-delimited JSON files with a stateful per-key count, executed via `Trigger.AvailableNow`, completed in 8.34 s end-to-

Table 3. Strong scaling, fixed 2 M-row job.

Workers (<code>local [k]</code>)	Median time (s)	Speed-up
1	0.604	1.00 $\times$
2	0.570	1.06 $\times$
4	0.593	1.02 $\times$

**Figure 3.** Measured speed-up vs. the linear ideal. The 2-core host bounds attainable scaling.

end—about **11,989 events/s** including query planning and micro-batch scheduling.

Auto-grader latency. Fig. 4 and Table 4 decompose interactive latency. Pure-Python challenges are graded in ~ 1 ms. Spark challenges incur a one-time JVM/session cold start of ~ 3.5 s; a full Spark grade (start + execute + check + teardown) completes in ~ 8.6 s. The warm-up dominates, which directly motivates the Spark Connect warm-pool design noted in §A: a shared session would remove the 3.5 s term from every interaction.

VII. THREATS TO VALIDITY

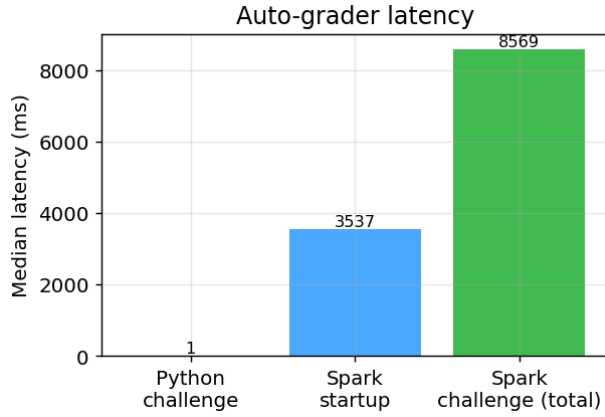
Performance results are hardware-dependent and were gathered on a 2-vCPU host; the flat throughput and limited speed-up reflect small, overhead-bound jobs rather than a limitation of Spark, which scales to far larger data and clusters. We measure *system* performance, not learning outcomes; a user study with pre/post assessments and engagement metrics is future work. Finally, the security analysis is qualitative—the current sandbox targets a trusted, single-tenant demo, and hostile-multi-tenant deployment requires the containerized isolation we describe.

VIII. FUTURE WORK

Planned extensions include: a warm Spark Connect pool to remove cold-start latency; persistent progress and authentication; a randomized-input grading mode to deter hard-coding; a controlled study of learning gains and retention; and additional tracks (Delta Lake, partitioning/shuffle tuning, and a capstone ETL project on a real Kaggle dataset).

Table 4. Median auto-grader latency.

Operation	Median latency
Python challenge (run + grade)	1 ms
Spark session cold start	3,537 ms
Spark challenge (end-to-end)	8,569 ms

**Figure 4.** Auto-grader latency. The Spark cold start is the dominant, and removable, cost.

IX. CONCLUSION

SparkQuest shows that a beginner can write and execute *real* distributed Spark code within minutes, with trustworthy automated feedback and game mechanics that sustain progress. The system pairs a careful execution-and-grading design with a complete, reproducible engineering pipeline—tests, CI, containerization, and free live deployment. On a deliberately constrained host it sustains multi-million row aggregation and $\sim 12k$ events/s streaming, grades Python instantly, and isolates a removable Spark cold start as its dominant interactive cost. The project is open source and live; we hope it lowers the on-ramp to data engineering.

REFERENCES

- [1] M. Zaharia *et al.*, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proc. NSDI*, 2012.
- [2] M. Zaharia *et al.*, “Apache Spark: A unified engine for big data processing,” *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [3] M. Armbrust *et al.*, “Spark SQL: Relational data processing in Spark,” in *Proc. ACM SIGMOD*, 2015.
- [4] M. Armbrust *et al.*, “Structured Streaming: A declarative API for real-time applications in Apache Spark,” in *Proc. ACM SIGMOD*, 2018.
- [5] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in *Proc. OSDI*, 2004.
- [6] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, “From game design elements to gamefulness: Defining gamification,” in *Proc. MindTrek*, 2011.
- [7] J. Hamari, J. Koivisto, and H. Sarsa, “Does gamification work? A literature review of empirical studies on gamification,” in *Proc. HICSS*, 2014.
- [8] J. C. Paiva, J. P. Leal, and Á. Figueira, “Automated assessment in computer science education: A state-of-the-art review,” *ACM Trans. Comput. Educ.*, vol. 22, no. 3, 2022.
- [9] S. H. Edwards, “Using software testing to move students from trial-and-error to reflection-in-action,” in *Proc. ACM SIGCSE*, 2004.
- [10] C. Wilcox, “The role of automation in undergraduate computer science education,” in *Proc. ACM SIGCSE*, 2015.
- [11] M. Kölling, “The Greenfoot programming environment,” *ACM Trans. Comput. Educ.*, vol. 10, no. 4, 2010.
- [12] S. Ramírez, “FastAPI,” 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [13] L. Biewald, “Experiment tracking with Weights and Biases,” 2020. [Online]. Available: <https://wandb.ai>